

Query Optimization: Q & A

1 Algebraic Transformations

Relational Algebra Equivalences

Show that the following equivalences hold and explain how they can be applied to improve the efficiency of certain queries:

- $E_1 \bowtie_{\theta} (E_2 - E_3) \equiv (E_1 \bowtie_{\theta} E_2 - E_1 \bowtie_{\theta} E_3)$
- $\sigma_{\theta}(A\gamma_F(E)) \equiv A\gamma_F(\sigma_{\theta}(E))$, where θ uses only attributes from A .
- $\sigma_{\theta}(E_1 \bowtie E_2) \equiv \sigma_{\theta}(E_1) \bowtie E_2$, where θ uses only attributes from E_1 .

Solution

a. Proof: $E_1 \bowtie_{\theta} (E_2 - E_3) \equiv (E_1 \bowtie_{\theta} E_2 - E_1 \bowtie_{\theta} E_3)$

Let $R_1 = E_1 \bowtie_{\theta} (E_2 - E_3)$, $R_2 = E_1 \bowtie_{\theta} E_2$, and $R_3 = E_1 \bowtie_{\theta} E_3$.

- If a tuple $t \in R_1$, then its component from E_2 is in E_2 and not in E_3 . Thus, $t \in R_2$ and $t \notin R_3$. Therefore, $t \in (R_2 - R_3)$.
- If a tuple $t \in (R_2 - R_3)$, then $t \in R_2$ (matching E_1 and E_2) and $t \notin R_3$. Since $t \notin R_3$, its component $t[E_2\text{'s attributes}]$ cannot be in E_3 . Thus, $t[E_2] \in (E_2 - E_3)$, so $t \in R_1$.

Efficiency: The LHS is often more efficient because $(E_2 - E_3)$ reduces the operand size before the join. The RHS performs a join on the full E_2 , producing many extra tuples that are eventually discarded.

b. Proof: $\sigma_{\theta}(A\gamma_F(E)) \equiv A\gamma_F(\sigma_{\theta}(E))$ (where $\theta \subseteq A$)

Since θ only involves grouping attributes A , the selection condition σ_{θ} is independent of the aggregation results F .

- If a group's value for A is filtered out by σ_{θ} , then all tuples in that group in E would also be filtered out by $\sigma_{\theta}(E)$.
- Conversely, if a group is kept by σ_{θ} , the aggregation F will be the same regardless of whether the selection is applied before or after grouping, as no tuples within that group are discarded.

Efficiency: Evaluating the RHS is much better because it filters out tuples **before** performing the expensive aggregation process.

c. Proof: $\sigma_{\theta}(E_1 \bowtie E_2) \equiv \sigma_{\theta}(E_1) \bowtie E_2$ (where $\theta \subseteq E_1$)

Since θ only uses attributes from E_1 :

- A join result tuple (t_1, t_2) satisfies θ if and only if t_1 alone satisfies θ .

- Thus, if σ_θ is applied to the result, it is equivalent to only joining those t_1 tuples that already satisfy θ .

Efficiency: The RHS is significantly faster because it reduces the size of E_1 before the join operation, minimizing the number of tuple comparisons and intermediate result size.

Counter-examples for Non-equivalence

For each of the following pairs of expressions, give instances of relations that show the expressions are not equivalent.

- $\Pi_A(r - s)$ and $\Pi_A(r) - \Pi_A(s)$.
- $\sigma_{B < 4}(A \gamma_{\max(B) \text{ as } B}(r))$ and $A \gamma_{\max(B) \text{ as } B}(\sigma_{B < 4}(r))$.
- In the preceding expressions, if both occurrences of $\max$ were replaced by $\min$, would the expressions be equivalent?
- $(r \bowtie_R s) \bowtie_R t$ and $r \bowtie_R (s \bowtie_R t)$ (i.e., natural right outer join is not associative).
- $\sigma_\theta(E_1 \bowtie_L E_2)$ and $E_1 \bowtie_L (\sigma_\theta(E_2))$, where θ uses only attributes from E_2 .

Solution

a. Projection vs. Difference: Let $r(A, B) = \{(1, 2)\}$ and $s(A, B) = \{(1, 3)\}$.

- **LHS:**

$$\Pi_A(r - s) = \Pi_A(\{(1, 2)\}) = \{(1)\}.$$

- **RHS:**

$$\Pi_A(r) - \Pi_A(s) = \{(1)\} - \{(1)\} = \emptyset.$$

Thus, the two expressions are not equivalent.

b. Selection vs. Max Aggregation: Let $r(A, B) = \{(1, 2), (1, 5)\}$.

- **LHS:** Grouping by A gives $\max(B) = 5$. Applying $\sigma_{B < 4}$ eliminates the tuple, so the result is $\emptyset$.
- **RHS:** Applying $\sigma_{B < 4}$ first yields $\{(1, 2)\}$. Aggregation then produces $\{(1, 2)\}$.

Hence, the expressions are not equivalent.

c. Replacing *max* with *min*: Yes, the expressions become equivalent.

- If the minimum value satisfies the selection, it appears in both expressions.
- If it does not, both expressions discard the group.

Therefore, replacing $\max$ with $\min$ restores equivalence.

d. Right Outer Join is Not Associative: Let

$$r(A, B) = \{(1, 2)\}, \quad s(B, C) = \{(2, 3)\}, \quad t(A, D) = \{(1, 4)\}.$$

- **LHS:** $(r \bowtie_R s) = \{(1, 2, 3)\}$. Joining with t produces $\{(1, 2, \text{null}, 4)\}$.

- **RHS:** $(s \bowtie_R t)$ introduces null-extended tuples earlier. Joining with r produces $\{(1, 2, 3, \text{null})\}$.

Since the results differ, right outer join is not associative.

e. **Selection and Left Outer Join:** Let

$$R(A, B) = \{(1, 2)\}, \quad S(A, C) = \{(2, 3)\}, \quad \theta : C = 1.$$

- **LHS:** $R \bowtie_L S = \{(1, 2, \text{null})\}$. Applying σ_θ removes the tuple, giving $\emptyset$.
- **RHS:** $\sigma_\theta(S) = \emptyset$. Then $R \bowtie_L \emptyset = \{(1, 2, \text{null})\}$.

Thus, the expressions are not equivalent.

Join Size Estimation & Strategy

Consider relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$, with primary keys A, C , and E , respectively. Assume $|r_1| = 1000$, $|r_2| = 1500$, and $|r_3| = 750$. Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$, and give an efficient strategy for computing the join.

Solution

Size Estimation: The size estimates are independent of the join order. Consider the order $((r_1 \bowtie r_2) \bowtie r_3)$:

- **Join $r_1 \bowtie r_2$:** Since C is the primary key of r_2 , each tuple in r_1 can match at most one tuple in r_2 . Thus, size $\leq |r_1| = 1000$.
- **Join with r_3 :** The intermediate result has a functional dependency on E (inherited from r_2). Since E is the primary key of r_3 , each tuple in the intermediate result matches at most one tuple in r_3 .
- **Final Size:** The result will have at most 1000 tuples.

Efficient Strategy: An efficient strategy involving indices and pipelining:

1. Create an index on attribute C for relation r_2 and on E for r_3 .
2. For each tuple t_1 in r_1 :
 - (a) Use the index on r_2 to look up (at most one) matching tuple t_2 .
 - (b) If found, use the value $t_2[E]$ to look up (at most one) matching tuple t_3 in r_3 using its index.
 - (c) If t_3 is found, output the combined tuple (t_1, t_2, t_3) .

This approach avoids creating temporary relations and reads r_1 only once (Pipelined evaluation).

General Join Size Estimation (No Primary Keys)

Consider relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$ with no primary keys (except the entire schema). Given:

- $|r_1| = 1000$, $|r_2| = 1500$, $|r_3| = 750$
- $V(C, r_1) = 900$, $V(C, r_2) = 1100$
- $V(E, r_2) = 50$, $V(E, r_3) = 100$

Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$ and give an efficient strategy for computing the join.

Solution

Size Estimation:

The estimated size is determined by calculating the average number of matching tuples.

Step 1: Join $r_1 \bowtie r_2$:

- For each tuple in r_1 , on average $\frac{|r_2|}{V(C, r_2)} = \frac{1500}{1100} = \frac{15}{11}$ tuples from r_2 will match.
- Intermediate size: $|r_1| \times \frac{15}{11} = 1000 \times \frac{15}{11} = \frac{15000}{11} \approx 1364$ tuples.

Step 2: Join with r_3 :

- For each tuple in the intermediate result, on average $\frac{|r_3|}{V(E, r_3)} = \frac{750}{100} = 7.5$ tuples from r_3 will match.
- Final size: $\frac{15000}{11} \times 7.5 = \frac{15000 \times 750}{11 \times 100} \approx 10,227$ tuples.

Efficient Strategy:

1. Join r_1 and r_2 first, since the intermediate result (≈ 1364 tuples) is comparable to the original relation sizes.
2. Then join this result with r_3 .

This order minimizes the size of intermediate relations, reducing I/O and computation costs.

B⁺ Tree Index and Negation

Suppose a B⁺-tree index on `building` is available on relation `department` and no other index is available. What is the best way to handle the following selections involving negation?

- a. $\sigma_{\neg(\text{building} < \text{'Watson'})}(\text{department})$
- b. $\sigma_{\neg(\text{building} = \text{'Watson'})}(\text{department})$
- c. $\sigma_{\neg(\text{building} < \text{'Watson'} \wedge \text{budget} < 50000)}(\text{department})$

Solution

a. $\neg(\text{building} < \text{'Watson'}) \equiv \text{building} \geq \text{'Watson'}$:

- Use the B^+ -tree index to locate the first tuple where $\text{building} = \text{'Watson'}$.
- Follow the leaf-level pointer chain to retrieve all subsequent tuples (since the index is sorted).

b. $\neg(\text{building} = \text{'Watson'}) \equiv \text{building} \neq \text{'Watson'}$:

- The index is **not useful** here, as inequality spans most of the relation.
- Perform a **sequential file scan**, selecting all tuples where $\text{building} \neq \text{'Watson'}$.

c. $\neg(A \wedge B) \equiv \neg A \vee \neg B$ (De Morgan's Law):

$$\neg(\text{building} < \text{'Watson'} \wedge \text{budget} < 50000) \equiv \text{building} \geq \text{'Watson'} \vee \text{budget} \geq 50000$$

- Use the B^+ -tree index to retrieve all tuples with $\text{building} \geq \text{'Watson'}$ (as in part a).
- For each retrieved tuple, also check if $\text{budget} \geq 50000$ (satisfies the OR condition).
- Additionally, perform a sequential scan for tuples with $\text{building} < \text{'Watson'}$ and $\text{budget} \geq 50000$.

Alternatively, if most tuples satisfy the condition, a full sequential scan may be more efficient.

Function-Based Join Optimization

Consider the query:

```
SELECT * FROM r, s
WHERE upper(r.A) = upper(s.A);
```

where `upper()` returns its input with all lowercase letters replaced by uppercase.

- What plan might a database system generate for this query?
- Some systems use a block nested-loop join, which can be inefficient. Explain how hash-join or merge-join can be used instead.

Solution

a. **Query Plan Analysis:**

- Create relations r and s with some tuples, then use EXPLAIN to view the chosen plan.
- Compare with a direct equality plan ($r.A = s.A$) to see differences in cost estimates.
- Some databases (e.g., PostgreSQL) may use merge-join on the function result;

others may fall back to nested-loop join with poor statistics.

b. Adapting Hash-Join and Merge-Join:

- **Hash Join:**
 - (a) Hash the *result* of $\text{upper}(r.A)$ for the build relation.
 - (b) Probe using $\text{upper}(s.A)$ from the probe relation.
- **Merge Join:**
 - (a) Sort both relations on $\text{upper}(A)$ (not the raw attribute).
 - (b) Perform the standard merge algorithm on the sorted results.
- **Index Support:** A **functional index** on $\text{upper}(A)$ would allow index nested-loop joins.

The key insight is that the join algorithms work unchanged once the function is applied to the join attribute.

Pushing Aggregation Through Join

Give conditions under which the following expressions are equivalent:

$$A,B\gamma_{\text{agg}(C)}(E_1 \bowtie E_2) \equiv (A\gamma_{\text{agg}(C)}(E_1)) \bowtie E_2$$

where agg denotes any aggregation operation. How can these conditions be relaxed if agg is $\min$ or $\max$?

Solution

General Conditions for Equivalence:

The expressions are equivalent if:

1. E_2 contains only attributes A and B .
2. A is the **primary key** of E_2 (no duplicate A values).

Reasoning:

- If there are duplicate values in $E_2.A$, the LHS would aggregate over multiple copies of the same E_1 tuples (due to the join), producing different results than the RHS.
- It is acceptable if E_2 does not contain some A values that exist in E_1 —such values get filtered out in both expressions.

Relaxation for $\min/\max$:

- Duplicate A values in E_2 do **not affect** $\min/\max$ results (since the extreme value remains the same regardless of repetition).
- However, there must be **no duplicates on (A, B)** in E_2 . The LHS (aggregation after join) would remove such duplicates, while the RHS (aggregation before join) would not.

Interesting Orders in Optimization

Consider a query that computes the natural join of a set of relations S . Given a subset $S_1 \subseteq S$, what are the **interesting orders** of S_1 ?

Solution

Definition of Interesting Orders:

An **interesting order** is a sort order on a set of attributes that could make a future operation (join, group by, order by) cheaper.

For a subset S_1 of S :

1. Let T be the set of all attributes of S_1 that **also occur** in any relation in $S - S_1$.
2. These are the attributes that will participate in future join conditions.
3. Every ordering of every **subset of** T is an interesting order.

Reasoning:

- If the intermediate result of S_1 is sorted on attributes in T , a future merge-join with relations in $S - S_1$ may become cheaper.
- The optimizer must track all such orders to make optimal decisions about join algorithms.

Number of Join Orders (Catalan Numbers)

Show that with n relations, there are $\frac{(2(n-1))!}{(n-1)!}$ different join orders.

Hint: A complete binary tree has every internal node with exactly two children. The number of complete binary trees with n leaf nodes is:

$$\frac{1}{n} \binom{2(n-1)}{n-1}$$

This is the **Catalan number** C_{n-1} .

Solution

Proof:

Step 1: Join Orders as Complete Binary Trees

- Each join order corresponds to a **complete binary tree** where the n relations are leaves.
- Every internal node represents a join operation with exactly two children.

Step 2: Bijection with Binary Trees

- A complete binary tree with n leaves has exactly $n - 1$ internal nodes.
- There is a bijection between complete binary trees with n leaves and binary trees with $n - 1$ nodes:

1. Remove all leaves from a complete binary tree $\Rightarrow$ binary tree with $n - 1$ nodes.
2. Conversely, add leaves to any binary tree with $n - 1$ nodes in a unique way $\Rightarrow$ complete binary tree.

Step 3: Catalan Number

- The number of binary trees with $n - 1$ nodes is the Catalan number:

$$C_{n-1} = \frac{1}{n} \binom{2(n-1)}{n-1}$$

Step 4: Labeled Trees (Permutations)

- The leaves (relations) can be arranged in $n!$ ways.
- Total join orders $= C_{n-1} \times n! = \frac{1}{n} \binom{2(n-1)}{n-1} \times n! = \frac{(2(n-1))!}{(n-1)!}$

DP Complexity: $O(3^n)$ Time Bound

Show that the lowest-cost join order can be computed in time $O(3^n)$. Assume that storing and looking up information about a set of relations takes constant time.

(If difficult, show the looser bound of $O(2^{2n})$.)

Solution

Proof using Binomial Expansion:

Consider the dynamic programming algorithm for join order optimization.

Step 1: Cost per Subset

- For each subset S of size $k + 1$, we consider all ways to partition S into two non-empty subsets A and $B = S - A$.
- For a fixed S , the number of such partitions is at most $2^{k+1} - 2$ (excluding $\emptyset$ and S).
- Each partition comparison takes $O(1)$ time (assuming constant-time lookup).

Step 2: Total Cost

- There are $\binom{n}{k+1}$ subsets of size $k + 1$.
- Cost for all subsets of size $k + 1$: $\binom{n}{k+1} \cdot 2^{k+1}$

Step 3: Summing Over All Sizes

$$\text{Total cost} = \sum_{k=1}^{n-1} \binom{n}{k+1} \cdot 2^{k+1} = \sum_{j=2}^n \binom{n}{j} \cdot 2^j$$

Using the binomial theorem with $x = 2$:

$$(1 + 2)^n = \sum_{j=0}^n \binom{n}{j} \cdot 2^j = 3^n$$

Since our sum excludes $j = 0$ and $j = 1$:

$$\sum_{j=2}^n \binom{n}{j} \cdot 2^j = 3^n - 1 - 2n < 3^n$$

Conclusion: The total time complexity is $O(3^n)$.

Left-Deep Join Trees: $O(n \cdot 2^n)$ Complexity

Show that if only **left-deep join trees** are considered (as in the System R optimizer), the time to find the most efficient join order is approximately $n \cdot 2^n$. Assume there is only one interesting sort order.

Solution

Proof:

The derivation is similar to the general case, but with a key restriction.

Step 1: Left-Deep Constraint

- In a left-deep tree, the **right operand** of the topmost join must be a **single relation**.
- For a subset S of size $k + 1$, we only consider $k + 1$ choices (one for each relation that could be on the right).

Step 2: Cost per Subset Size

- There are $\binom{n}{k+1}$ subsets of size $k + 1$.
- Each requires $k + 1$ comparisons.
- Total cost for size $k + 1$: $\binom{n}{k+1} \cdot (k + 1) = n \cdot \binom{n-1}{k}$

Step 3: Summing Over All Sizes

$$\text{Total cost} = \sum_{k=1}^{n-1} n \cdot \binom{n-1}{k} = n \cdot \sum_{k=1}^{n-1} \binom{n-1}{k}$$

Using the binomial theorem with $x = 1$:

$$\sum_{k=0}^{n-1} \binom{n-1}{k} = 2^{n-1}$$

Excluding $k = 0$:

$$\sum_{k=1}^{n-1} \binom{n-1}{k} = 2^{n-1} - 1 < 2^{n-1}$$

Conclusion: Total time $< n \cdot 2^{n-1} = O(n \cdot 2^n)$.

Scalar Aggregate Decorrelation (Bank Database)

Consider a bank database with schema:

`account(account_number, branch_name, balance)`

- Write a nested query to find, for each branch with name starting with 'B', all accounts with the maximum balance at that branch.
- Rewrite the query without a nested subquery (decorrelate it in SQL).
- Give a relational algebra expression using **semijoin** equivalent to the query.
- Provide a general procedure for decorrelating such scalar aggregate queries.

Solution

a. Nested Query:

```
SELECT S.account_number
FROM account S
WHERE S.branch_name LIKE 'B%' AND
      S.balance = (SELECT MAX(T.balance)
                  FROM account T
                  WHERE T.branch_name = S.branch_name);
```

b. Decorrelated SQL Query:

```
-- Step 1: Compute group maxima
CREATE TABLE t1 AS
  SELECT branch_name, MAX(balance) AS max_bal
  FROM account
  GROUP BY branch_name;

-- Step 2: Join with original relation
SELECT account.account_number
FROM account, t1
WHERE account.branch_name LIKE 'B%' AND
      account.branch_name = t1.branch_name AND
      account.balance = t1.max_bal;
```

c. Relational Algebra with Semijoin:

Let $T = \text{branch_name} \gamma_{\text{max}(\text{balance}) \text{ as max_bal}}(\text{account})$.
Then:

$$\Pi_{\text{account_number}} \left(\begin{array}{l} \sigma_{\text{branch_name LIKE 'B\%'}}(\text{account}) \\ \bowtie_{\text{branch_name, balance=max_bal}} T \end{array} \right)$$

d. General Decorrelation Procedure:

For queries of the form:

```
SELECT L1 FROM R WHERE P1 AND A1 op (SELECT f(A2) FROM S WHERE P2)
```

1. **Separate predicates:** Split P_2 into P_2^1 (independent) and P_2^2 (correlated).
2. **Create grouped subquery:**

```
CREATE TABLE t1 AS
  SELECT f(A2) AS agg_result, V
  FROM S WHERE P2_1 GROUP BY V;
```

where V = attributes used in correlation predicates.

3. **Join with original:**

```
SELECT L1 FROM R, t1
WHERE P1 AND P2_2 AND A1 op t1.agg_result;
```

Composite B⁺-Tree Index Selection

Suppose a B⁺-tree index on (dept_name, building) is available on relation department. What is the best way to handle the following selection?

$$\sigma(\text{building} < \text{'Watson'}) \wedge (\text{budget} < 55000) \wedge (\text{dept_name} = \text{'Music'}) (\text{department})$$

Solution

Index Structure Analysis:

- The composite index is ordered by (dept_name, building).
- The index can efficiently handle predicates on dept_name (the first key) and building (the second key, given a fixed dept_name).
- The budget predicate **cannot** be handled by this index.

Optimal Strategy:

1. **Use the index** to locate all entries where:
 - dept_name = 'Music' (exact match on first key)
 - building < 'Watson' (range scan on second key)
2. The index finds the first entry with dept_name = 'Music' and building = min.
3. Follow the leaf-level pointer chain until building >= 'Watson' or dept_name changes.
4. For each retrieved tuple, **apply the residual predicate:** budget < 55000.

Why This Works:

- The composite index groups all 'Music' entries together, sorted by building.
- The range condition $\text{building} < \text{'Watson'}$ defines a contiguous range within the 'Music' entries.
- Only the budget condition requires a post-fetch filter.

Deriving Equivalences by Transformation

Show how to derive the following equivalences by a sequence of transformations using equivalence rules:

- $\sigma_{\theta_1 \wedge \theta_2 \wedge \theta_3}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(\sigma_{\theta_3}(E)))$
- $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta_3} E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta_3} (\sigma_{\theta_2}(E_2)))$, where θ_2 involves only attributes from E_2 .

Solution

a. Cascading Selections:

Using the **Selection Cascade Rule**: $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$

Derivation:

$$\begin{aligned}\sigma_{\theta_1 \wedge \theta_2 \wedge \theta_3}(E) &\equiv \sigma_{\theta_1 \wedge \theta_2}(\sigma_{\theta_3}(E)) && \text{(Apply cascade rule once)} \\ &\equiv \sigma_{\theta_1}(\sigma_{\theta_2}(\sigma_{\theta_3}(E))) && \text{(Apply cascade rule again)}\end{aligned}$$

b. Pushing Selection Through Join:

Using the rules:

- **Selection Cascade**: $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
- **Selection Push-down**: If θ_2 involves only attributes of E_2 , then $\sigma_{\theta_2}(E_1 \bowtie_{\theta_3} E_2) \equiv E_1 \bowtie_{\theta_3} (\sigma_{\theta_2}(E_2))$

Derivation:

$$\begin{aligned}\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta_3} E_2) &\equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E_1 \bowtie_{\theta_3} E_2)) && \text{(Selection cascade)} \\ &\equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta_3} (\sigma_{\theta_2}(E_2))) && \text{(Push } \theta_2 \text{ into } E_2)\end{aligned}$$

The final step is valid because θ_2 involves only attributes from E_2 .

Selection on Left Outer Join vs. Natural Join

Consider the two expressions $\sigma_{\theta}(E_1 \bowtie_L E_2)$ and $\sigma_{\theta}(E_1 \bowtie E_2)$.

- Show using an example that the two expressions are **not equivalent** in general.
- Give a simple condition on the predicate θ that ensures the two expressions **are equivalent**.

Solution

a. Counter-example:

Let $E_1(A, B) = \{(1, 2)\}$ and $E_2(A, C) = \{(2, 3)\}$. Let $\theta : A = 1$.

Left Outer Join:

- $E_1 \bowtie_L E_2 = \{(1, 2, \text{null})\}$ (no match on A , so C is null-padded).
- $\sigma_{A=1}(\dots) = \{(1, 2, \text{null})\}$.

Natural Join:

- $E_1 \bowtie E_2 = \emptyset$ (no matching A values).
- $\sigma_{A=1}(\emptyset) = \emptyset$.

The results differ: $\{(1, 2, \text{null})\} \neq \emptyset$.

b. Condition for Equivalence:

The expressions are equivalent if θ involves **only attributes from E_2** and θ **rejects null values**.

Reasoning:

- If θ involves an attribute from E_2 that is null-padded (due to no match), θ will evaluate to UNKNOWN or FALSE.
- Such tuples are filtered out by σ_θ , leaving only tuples that also appear in the natural join.
- Thus, if θ involves E_2 attributes and rejects nulls, both expressions produce the same result.

Histogram-Based Selection Estimation

Explain how to use a histogram to estimate the size of a selection of the form $\sigma_{A \leq v}(r)$.

Solution

Histogram Structure:

A histogram divides the range of attribute A into **buckets**. Each bucket records:

- The range of values it covers: $[l_i, h_i]$
- The number of tuples in that range: n_i

Estimation Procedure for $\sigma_{A \leq v}(r)$:

1. Identify relevant buckets:

- All buckets where $h_i \leq v$ are **fully included**.
- The bucket where $l_i \leq v < h_i$ is **partially included**.

2. **Sum fully included buckets:**

$$\text{Full count} = \sum_{h_i \leq v} n_i$$

3. **Estimate partial bucket contribution:**

- Assume uniform distribution within the bucket.
- Fraction included: $\frac{v - l_i}{h_i - l_i}$
- Partial count: $n_i \times \frac{v - l_i}{h_i - l_i}$

4. **Total estimate:**

$$|\sigma_{A \leq v}(r)| \approx \text{Full count} + \text{Partial count}$$

Example:

Histogram with buckets: $[0, 10] : 100$, $[10, 20] : 150$, $[20, 30] : 200$.

For $\sigma_{A \leq 15}(r)$:

- Full buckets: $[0, 10] \rightarrow 100$ tuples.
- Partial bucket $[10, 20]$: $150 \times \frac{15-10}{20-10} = 150 \times 0.5 = 75$.
- Estimate: $100 + 75 = 175$ tuples.

Join Size Estimation with Histograms

Suppose two relations r and s have histograms on attributes $r.A$ and $s.A$, respectively, but with different ranges. Suggest how to use the histograms to estimate the size of $r \bowtie s$.

Solution

To estimate the size of $r \bowtie s$ using histograms with different ranges and bucket boundaries, we use a technique called **histogram alignment** by splitting the ranges of each histogram.

Step 1: Define Refined Boundaries

- Identify the overlapping range $[L, R]$ of the two histograms: $L = \max(\min(r.A), \min(s.A))$ and $R = \min(\max(r.A), \max(s.A))$.
- Collect all bucket boundaries from both histograms that fall within $[L, R]$.
- Sort these boundaries to create a set of **refined intervals** $[b_i, b_{i+1}]$.

Step 2: Redistribute Tuples into Refined Intervals For each refined interval $[b_i, b_{i+1}]$, estimate the number of tuples from r ($n_{r,i}$) and s ($n_{s,i}$) that fall into it:

- Use linear interpolation: if a refined interval is a fraction of an original bucket, assume tuples are distributed uniformly.
- $n_{r,i} = (\text{tuples in original bucket}) \times \frac{b_{i+1} - b_i}{\text{width of original bucket}}$.

Step 3: Estimate Local Join Size Within each refined interval i , assume uniform distribution. The join size contribution is:

$$\text{JoinSize}_i = \frac{n_{r,i} \times n_{s,i}}{\max(V(A, r_i), V(A, s_i))}$$

where $V(A, r_i)$ and $V(A, s_i)$ are the estimated number of distinct values in the interval for r and s respectively.

- If $V(A)$ is not known per bucket, we can estimate it as: $V(A, r_i) = V(A, r) \times \frac{b_{i+1} - b_i}{\max(r.A) - \min(r.A)}$ (or simply assume it is proportional to the number of tuples).

Step 4: Total Estimate The total join size is the sum of contributions from all refined intervals:

$$|r \bowtie s| \approx \sum_i \text{JoinSize}_i$$

Query Decorrelation with Multiset Semi-Join

Consider the query:

```
select A, B
from r
where r.B < some (select B
                  from s
                  where s.A = r.A)
```

Show how to decorrelate this query using the multiset version of the semi join operation.

Solution

The query expression involves a correlated subquery in the where clause with a $<$ some comparison.

1. Analysis of the Original Query: The selection condition is $\sigma_{r.B < \text{some } \mathcal{E}(r)}$, where the subquery $\mathcal{E}(r)$ is $\Pi_{s.B}(\sigma_{s.A=r.A}(s))$. The condition $r.B < \text{some } (\dots)$ is satisfied if there exists at least one tuple in the subquery result such that $r.B$ is less than its B value.

2. Transformation to Semi-Join: The existence check combined with the correlation condition $s.A = r.A$ can be expressed as a semi-join.

- The correlation condition is: $r.A = s.A$.
- The inner selection/comparison condition is: $r.B < s.B$.

Combining these, we get the join predicate $C = (r.A = s.A \wedge r.B < s.B)$.

3. Multiset Version: Since the original query uses `select A, B` (without `distinct`), it preserves the multiplicity of tuples in r that satisfy the condition. The **multiset semi-join** ($\bowtie_{\text{multiset}}$) is defined precisely to return each tuple in r as many times as it appears in r , provided it has at least one matching tuple in s .

4. Decorrelated Expression: The decorrelated relational algebra expression is:

$$r \bowtie_{r.A=s.A \wedge r.B < s.B} s$$

where $\bowtie$ denotes the multiset version of the semi-join.

Incremental Maintenance of Views

Describe how to incrementally maintain the results of the following operations on both insertions and deletions:

- Union and set difference.
- Left outer join.

Solution

Incremental maintenance involves updating the result of a query given changes (insertions i_r or deletions d_r) to the base relations r and s .

a. Union and Set Difference

- **Union ($r \cup s$):**
 - **Insertion (i_r):** Add i_r to the result. If using set semantics, only add tuples that are not already in s .
 - **Deletion (d_r):** Remove d_r from the result. If using set semantics, only remove tuples that are not in s .
- **Set Difference ($r - s$):**
 - **Insertion (i_r):** Add tuples in i_r that are **not** in s .
 - **Deletion (d_r):** Remove tuples in d_r from the result.
 - **Insertion (i_s):** Remove tuples from the result that are in i_s .
 - **Deletion (d_s):** Add tuples to the result that are in d_s provided they are present in r .

b. Left Outer Join ($r \ltimes s$) Maintaining outer joins is more complex because of null-padding. Let $r \ltimes s = (r \bowtie s) \cup (r - \Pi_R(r \bowtie s)) \times \{(\text{null}, \dots, \text{null})\}$.

- **Changes to r :**
 - **Insertion (i_r):** For each tuple $t \in i_r$, join it with s . If it has matches, add those joined tuples. If it has **no** matches, add t padded with nulls.
 - **Deletion (d_r):** Simply remove all tuples in the result that were derived from d_r (both joined and null-padded).
- **Changes to s :**
 - **Insertion (i_s):** Join i_s with r . For every matching tuple $t_r \in r$:

1. If t_r previously had **no** matches in s , it existed in the result as a null-padded tuple. Remove that null-padded tuple.
 2. Add the new joined tuples (t_r, t_s) .
- **Deletion (d_s):** Join d_s with r . For every matching tuple $t_r \in r$:
1. Remove the joined tuples.
 2. Check if t_r still has any **other** matches in s . If this was the **last** match, add the null-padded tuple for t_r back to the result.

IVM vs. Full Recomputation

Give an example of an expression defining a materialized view and two situations (sets of statistics for the input relations and the differentials) such that incremental view maintenance is better than recomputation in one situation, and recomputation is better in the other situation.

Solution

Consider the materialized view V defined by the join of two relations:

$$V = r \bowtie s$$

Let r have schema $R(A, B)$ and s have schema $S(B, C)$. Assume an insertion Δr occurs in relation r .

Situation 1: Incremental Maintenance is Better

- **Statistics:**
 - $|r| = 10^7$ tuples, $|s| = 10^7$ tuples.
 - $|\Delta r| = 1$ tuple (a single new insertion).
- **Analysis:**
 - **Recomputation:** Requires joining 10^7 tuples of r with 10^7 tuples of s . Even with a hash join, the cost is proportional to $|r| + |s|$.
 - **Incremental:** The update is $\Delta V = \Delta r \bowtie s$. Since $|\Delta r| = 1$, this involves a single index lookup in s (if an index on B exists). The cost is $O(1)$ or $O(\log |s|)$, which is orders of magnitude faster.

Situation 2: Full Recomputation is Better

- **Statistics:**
 - $|r| = 10^5$ tuples, $|s| = 10^5$ tuples.
 - $|\Delta r| = 9 \times 10^4$ tuples (a bulk load replacing almost all of r).
- **Analysis:**

- **Incremental:** Requires joining 9×10^4 tuples with s . This might involve 9×10^4 separate index lookups or a large join operation. The overhead of managing the differential, transaction logging, and potentially updating many individual rows in the materialized view table can be high.
- **Recomputation:** A full recomputation can use highly optimized bulk operations like a sort-merge join or a linear-time hash join on the entire relations. For large batch updates, the per-tuple overhead of recomputation is often lower than the overhead of incremental maintenance logic.

Top-K Join Optimization

Suppose you want to get answers to $r \bowtie s$ sorted on an attribute of r , and want only the top K answers for some relatively small K . Give a good way of evaluating the query:

- When the join is on a foreign key of r referencing s , where the foreign key attribute is declared to be not null.
- When the join is not on a foreign key.

Solution

a. Foreign Key Join: Since the join is on a non-null foreign key of r referencing s , we know that **every** tuple in r will have exactly one matching tuple in s (if B is a primary key in s).

- **Evaluation strategy:** Use an index on the sort attribute of r to scan the first K tuples of r in the desired order.
- For each of these K tuples, perform an index-lookup join with s .
- Since every r tuple matches, we are guaranteed to have K join results after processing exactly K tuples of r .
- This avoids a full join and a full sort.

b. General Join: In a general join, some tuples of r may not have matches in s , or may have multiple matches.

- **Evaluation strategy:** Still scan r in the sorted order using an index.
- Perform an index-lookup join with s for each tuple t_r .
- Maintain a count of the number of join results produced.
- Stop as soon as the total number of result tuples reaches K .
- If the join selectivity is high, this will still be much faster than a full join, even if we have to scan slightly more than K tuples of r .

Index-Only Plans

Consider a relation $r(A, B, C)$, with an index on attribute A . Give an example of a query that can be answered by using the index only, without looking at the tuples in the relation.

Solution

An **index-only plan** is possible when all attributes required by the query (for selection, join, and projection) are present in the index itself.

Example Query:

```
SELECT count(A)
FROM r
WHERE A > 100 AND A < 200;
```

Explanation: The index on A contains all the values of A in sorted order. To answer this query, the system can:

1. Traverse the B+-tree index to find the first entry where $A > 100$.
2. Scan the leaf nodes of the index, incrementing a counter for each entry found, until $A \geq 200$.
3. Return the final count.

The actual data blocks (the heap file) of relation r are never accessed.

The Halloween Problem

Suppose you have an update query U . Give a simple sufficient condition on U that will ensure that the Halloween problem cannot occur, regardless of the execution plan chosen or the indices that exist.

Solution

The **Halloween Problem** occurs when an update of a tuple causes that tuple to be visited again during the same scan (e.g., if the update changes an attribute used by the index which is currently being used for the scan, moving the tuple "forward" in the scan order).

Sufficient Condition: The update will not suffer from the Halloween problem if the **set of attributes modified by the update is disjoint from the set of attributes used in the selection condition and the access path.**

More simply:

$$\text{ModifiedAttributes}(U) \cap \text{SearchAttributes}(U) = \emptyset$$

If the attributes being updated are not used to determine which tuples to retrieve or the order of retrieval (i.e., they are not part of the search key of any index used in the plan), the tuple's position in the scan order cannot change, and it will not be seen twice.